

An Intelligent Multi-Agent Framework of Code Analysis and Automated Code Refactoring

Abstract—Software codebases are difficult to modernize because accumulated technical debt, complex dependencies, code smells, and limited test coverage make refactoring both time-consuming and risky. Existing static analysis and integrated development environment tools can identify quality issues or execute local refactorings, while recent artificial intelligence based approaches can generate broader code changes. However, these approaches commonly separate code understanding, refactoring planning, behavior-preserving validation, and developer control. This paper presents an agentic intelligent code refactoring assistant that organizes these responsibilities into four cooperating agents. The Code Understanding and Quality Assessment agent analyzes a repository and produces structured quality evidence. The Refactoring Decision and Planning agent converts the detected issues into a prioritized, dependency-aware, machine-executable refactoring plan using rule-based reasoning, machine-learning-based prediction, and multi-criteria decision analysis. The Safe Code Transformation and Validation agent applies approved changes and evaluates syntax, structure, available tests, behavioral fingerprints, and mined invariants, with rollback when validation fails. The Developer Interaction and Workflow Orchestration (DIWO) agent governs three human-in-the-loop decision gates for code-smell selection, refactoring-plan approval, and final refactoring acceptance, while providing impact information, before–after comparison, workflow traceability, feedback capture, and auditable decision records. The framework is evaluated at component and end-to-end levels using Python, Java and C repositories with maintainability issues, code-quality measures, planning and validation metrics, and developer-oriented workflow measures.

Keywords—agentic software engineering, automated code refactoring, multi-agent systems, code quality, code smells, behavior-preserving transformation, refactoring validation, human-in-the-loop, software maintenance

I. INTRODUCTION

Software codebases remain important in many organizations, but long-term evolution often leaves them with duplicated logic, large classes, long methods, tight coupling, outdated structures, and limited automated tests. Refactoring is intended to improve the internal structure of software

without changing its externally observable behavior [1], [2]. In practice, however, the safety of a refactoring depends on how well the existing code is understood, whether a suitable transformation is selected, and whether the resulting program is adequately validated. These conditions are difficult to satisfy in large codebases where documentation and regression tests are incomplete.

Static analysis platforms and refactoring tools have reduced part of this effort. Tools such as SonarQube report maintainability issues and code smells, while integrated development environments provide operations such as Extract Method, Rename, and Move Method. These tools are useful for localized analysis and transformation, but their outputs still require developer interpretation and they do not provide a complete decision process that connects repository-level analysis to a safe sequence of transformations [1], [3]. Research on refactoring also shows that behavior preservation remains a central concern, particularly when transformations are applied automatically [4].

Recent work has explored search-based and artificial-intelligence-assisted refactoring. Multi-agent approaches such as RefactorGPT and RefAgent divide the task among specialized agents and demonstrate that agent collaboration can improve automated refactoring workflows [5], [6]. Nevertheless, an important gap remains between detecting a problem and safely accepting a transformed codebase. Existing approaches often embed planning decisions inside prompts, depend strongly on available tests, or provide limited developer visibility into intermediate decisions. This makes it difficult to reason about why a refactoring was selected, whether dependent refactorings are ordered correctly, and what evidence supports accepting the final change.

To address this gap, this study introduces a human-in-the-loop multi-agent framework for code refactoring. The framework decomposes the process into four responsibilities: code understanding and quality assessment, refactoring decision and

planning, safe transformation and validation, and developer interaction and workflow orchestration. Structured JSON contracts connect the stages so that each agent produces an explicit artifact for the next stage rather than passing unstructured instructions. The developer is kept in the workflow through approval gates before transformation and final acceptance.

The contribution of this work is fourfold. First, it converts static and structural code analysis into prioritized, machine-readable quality evidence for downstream agents. Second, it introduces a hybrid planning mechanism that combines rule-based refactoring knowledge, learned suitability and risk estimates, multi-criteria decision analysis, and dependency-aware sequencing. Third, it combines controlled source-code transformation with layered safety checks, including a test-less behavior-preservation mechanism based on behavioral fingerprinting and invariant mining. Finally, it integrates these capabilities through an approval-gated orchestration layer that prevents rejected artifacts from propagating to subsequent agents, preserves file-level traceability from detected code smells to validated transformations, and records developer decisions, validation evidence, feedback, and final outcomes for auditability.

II. RELATED WORK

A. Code Quality Analysis and Refactoring Support

Code smells are widely used as indicators of maintainability problems and refactoring opportunities. Fowler’s refactoring catalog established commonly used transformations and their relation to design problems [1]. Static analysis tools extend this idea by scanning large repositories for bugs, maintainability issues, and rule violations. SonarQube is an example of an industrial platform that provides automated quality inspection and maintainability reporting [3]. Although these tools support large-scale inspection, the resulting reports are primarily developer-facing and do not by themselves define an ordered refactoring strategy.

Automated refactoring research has also considered the accuracy of refactoring detection and the structure of transformations [7]. RefactoringMiner demonstrates that refactoring operations can be identified with high accuracy from source-code histories [8]. Other approaches formalize preconditions to determine whether a refactoring can be safely applied [9]. These techniques provide important building blocks, but they generally address specific parts of the refactoring process rather than the complete analysis–planning–validation–approval workflow considered in this study.

B. Planning and Multi-Agent Refactoring

Search-based and AI-assisted approaches have moved automated refactoring beyond isolated IDE operations. RefactorGPT uses multiple agents for analysis, refactoring, refinement, and error recovery, while RefAgent similarly applies a multi-agent design to automated software refactoring [5], [6]. These approaches show the usefulness of task decomposition, but planning is commonly coupled to generative decisions or iterative correction. The proposed Refactoring Decision and

Planning (RDP) agent instead treats planning as an explicit artifact-generation stage. Candidate refactorings are first derived from known smell–refactoring relationships, then evaluated according to suitability, expected quality improvement, behavioral risk, implementation complexity, and dependency constraints before an ordered plan is produced.

C. Behavior Preservation and Validation

Behavior preservation is a fundamental requirement of refactoring [2], [4]. Test execution is the most practical validation mechanism when a reliable regression suite exists, but codebases frequently contain incomplete or weak tests. Formal approaches can provide stronger guarantees but can be difficult to apply at repository scale. Dynamic techniques therefore provide a useful complementary source of evidence. Daikon demonstrated that likely program invariants can be inferred from executions [10], while metamorphic testing provides a way to reason about correctness when an explicit test oracle is difficult to define [11]. The Safe Code Transformation and Validation (SCTVA) agent builds on these ideas by comparing pre- and post-refactoring behavior signatures and mined invariants in addition to syntax, structure, static checks, and available project tests.

D. Developer Control, Workflow Fit, and Traceability

Automated refactoring can reduce repetitive work, but fully automatic changes create a trust problem when developers cannot inspect intermediate decisions or recover from an unsuitable transformation. The proposed Developer Interaction and Workflow Orchestration (DIWO) agent addresses this concern by making approval, rejection, modification, comparison, feedback, and rollback explicit workflow states. Rather than treating the user interface as a presentation layer only, DIWO records the decision path followed by the system, thereby providing an auditable link between detected issues, generated plans, applied transformations, validation results, and the developer’s final decision.

Table I contrasts representative tools with the proposed framework.

III. RESEARCH METHODOLOGY

This study follows a Design Science Research (DSR) methodology supported by experimental evaluation. DSR is appropriate because the primary research output is a software artifact designed to address a practical software-engineering problem, followed by demonstration and evaluation of the artifact [14]. The work therefore progresses through problem identification, objective definition, artifact design, implementation, demonstration, and evaluation.

A. System Architecture

The framework treats refactoring as an evidence-driven pipeline in which each agent performs a specialized responsibility and exchanges structured artifacts with the other components. DIWO operates as the orchestration layer between the developer and the technical agents, controlling the progression of artifacts through the refactoring lifecycle. Table II

TABLE I
COMPARISON WITH EXISTING REFACTORING APPROACHES

System	Documented Scope	Difference from the Proposed Framework
SonarQube [3]	Static metrics and quality gates	Adds ranked planning, approved transformation, validation, rollback, and audit.
RefactoringMiner [8]	History-based detection and AST diffs	Plans and executes new refactorings rather than identifying completed changes.
OpenRewrite [12]	Recipe-based automated source transformation	Adds evidence ranking, approval gates, layered validation, and audit.
IntelliJ IDEA [13]	IDE refactorings, previews, and conflict handling	Adds a repository-level multi-agent lifecycle and audit trail.
RefactorGPT and RefAgent [5], [6]	Multi-agent LLM refactoring and iterative correction	Adds an explicit rule/ML/MCDA plan, three developer gates, test-poor validation, rollback, and traceable handoffs.
Proposed Framework	End-to-end four-agent refactoring pipeline	Integrates the above features through traceable, approval-controlled handoffs.

summarizes the responsibilities and outputs of the four agents.

The process begins when a developer provides a source

repository or project archive. CUQA analyzes the source code and produces a structured quality report containing detected code smells, affected files or source entities, severity information, software metrics, and supporting structural evidence. This report is first transferred to DIWO rather than being forwarded directly to RDP. DIWO presents the detected smells and their corresponding impact information to the developer and records an approve or reject decision for each finding. Only the developer-approved code smells are included in the request submitted by DIWO to RDP, while rejected findings are retained in the workflow history for traceability.

RDP generates candidate refactoring methods and an ordered refactoring plan for the approved smells. The generated plan is returned to DIWO, where the selected refactoring method, expected impact, planning information, and affected source elements are presented to the developer. The developer can approve or reject the proposed planning actions, and DIWO forwards only the approved subset of the refactoring plan to SCTVA. Consequently, a planning action rejected by the developer cannot propagate to the transformation stage.

After SCTVA performs the approved transformations and completes the associated validation activities, the transformed source files and validation evidence are returned to DIWO. DIWO presents the project through a status-aware repository hierarchy together with before–after source-code comparisons, allowing the developer to inspect the files affected throughout the workflow and make the final acceptance or rejection decision. Files associated with rejected smells, rejected planning actions, and completed refactorings remain distinguishable within the workflow view. When the result is accepted, DIWO makes the accepted project available for final artifact delivery and records the complete sequence of agent outputs and developer decisions in the audit trail.

B. Code Understanding and Quality Assessment

CUQA performs repository-level analysis using a combination of static analysis and rule-based source-code inspection. The repository acquisition stage identifies supported source files and prepares them for analysis. Structural information is extracted through language-aware parsing and Abstract Syntax

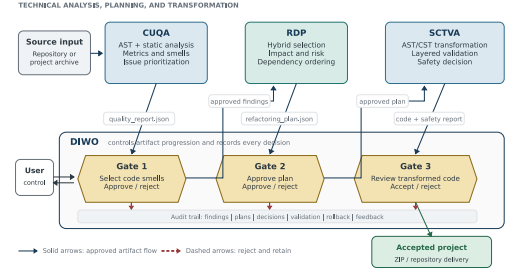


Fig. 1. End-to-end architecture and approval-controlled data flow of the proposed multi-agent refactoring framework. DIWO prevents rejected artifacts from propagating downstream while retaining decisions and evidence in the audit trail.

Tree (AST) analysis. Software-quality signals such as method size, class size, parameter count, duplication, cyclomatic complexity, coupling-related information, and rule violations are combined with detected smells.

The agent does not treat every warning as equally important. Detected issues are aggregated and prioritized according to severity, frequency, complexity, and potential maintainability impact. The final result is encoded as a machine-readable report that identifies the affected source element, smell type, supporting metric values, and priority information. This structured representation enables the planning stage to operate on explicit program evidence rather than a human-oriented dashboard.

C. Refactoring Decision and Planning

RDP converts the CUQA report into a structured refactoring plan. First, detected smells are mapped to candidate transformations using a refactoring knowledge base derived from established refactoring practices [1]. Candidate actions are then filtered using applicability conditions so that unsuitable transformations are removed before ranking.

For each remaining candidate, structural features such as complexity, coupling, cohesion, maintainability, source size, and dependency information are used to estimate contextual suitability, expected quality improvement, and behavioral risk. The proposal adopts a hybrid decision model in which rule-based reasoning provides deterministic candidate generation, a machine-learning component predicts candidate-level outcomes, and Multi-Criteria Decision Analysis (MCDA) com-

TABLE II
RESPONSIBILITIES AND DATA FLOW OF THE MULTI-AGENT FRAMEWORK

Agent	Primary Input	Main Responsibility	Primary Output
CUQA	Source repository and analysis settings	Parse source code, collect structural and quality metrics, detect code smells, aggregate findings, and prioritize maintainability issues	Structured quality report
RDP	CUQA quality report	Generate candidate refactorings, estimate expected impact and risk, rank alternatives, resolve dependencies, and build an ordered execution plan	Machine-executable refactoring plan
SCTVA	Approved plan and source files	Apply source transformations; validate syntax, structure, tests, behavior, and invariants; compute safety evidence; rollback unsafe changes	Validated refactored code and safety report
DIWO	CUQA findings, RDP plans, SCTVA results, and developer decisions	Enforce three approval gates; present impact and file-level workflow status; manage rejection filtering, before-after comparison, feedback, final artifact delivery, and audit reporting	Approved downstream artifacts, final decision, feedback records, and audit report

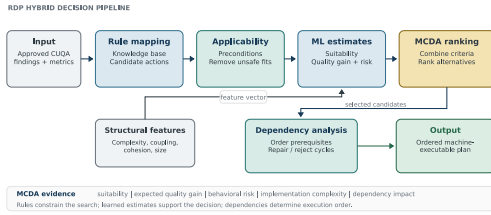


Fig. 2. RDP hybrid decision pipeline combining deterministic candidate generation, applicability filtering, learned suitability, quality-impact and risk estimates, MCDA ranking, and dependency-aware execution ordering.

bin these predictions with implementation complexity and dependency impact. This reduces dependence on a single probabilistic model while retaining data-driven decision support.

The selected refactorings are subsequently passed through dependency analysis. Operations that require another structural change are ordered before execution, and incompatible or circular plans are rejected or repaired. The final plan contains target elements, refactoring types, required parameters, dependency order, expected constraints, explanation fields, and validation checkpoints. This plan acts as the contract between planning and transformation.

D. Safe Code Transformation and Validation

SCTVA receives the approved machine-executable plan and applies the required transformations using AST/CST-aware techniques. The original source is preserved before modification so that the system can return to a known state when validation fails. The validation process is layered because no single check can establish behavior preservation in all projects.

The first layer verifies syntax and structural consistency after transformation. Static analysis and type-related checks are then used to detect newly introduced inconsistencies. When a project contains an automated test suite, tests are executed and compared across the original and transformed versions. For test-poor projects, the agent performs behavioral fingerprinting

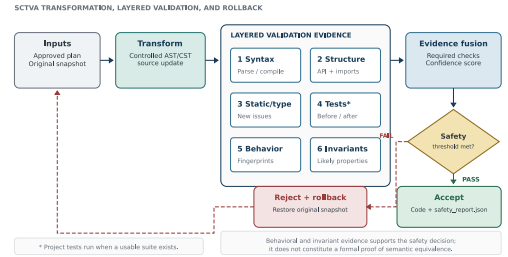


Fig. 3. SCTVA transformation and layered validation workflow. Failure of a required check or an insufficient confidence score rejects the transformation and restores the preserved original snapshot. Behavioral fingerprints and mined invariants provide supporting evidence rather than a formal proof of semantic equivalence.

by invoking selected functions with sampled inputs and recording observable signatures such as return values, return types, exception behavior, and execution outcomes. Invariant mining derives likely properties such as type stability, non-nullness, value ranges, execution-success consistency, and exception-pattern consistency. These runtime properties are compared before and after refactoring using principles related to dynamic invariant detection and oracle-light validation [10], [11].

A safety decision combines the available validation evidence into a confidence assessment. If required validation layers fail or the confidence threshold is not satisfied, the transformation is rejected and the original code is restored. Otherwise, SCTVA produces the transformed source together with a validation report describing the evidence used to support the decision.

E. Developer Interaction and Workflow Orchestration

DIWO coordinates the refactoring lifecycle through an approval-driven state model and acts as the human-in-the-loop orchestration layer between CUQA, RDP, and SCTVA. Three developer decision points are introduced: code-smell selection, refactoring-plan approval, and final refactoring acceptance. At the first stage, detected code smells and their impact

information are presented to the developer, and only approved findings are forwarded to RDP. The generated refactoring plan and its expected impacts are then presented for a second approval, after which only approved planning actions are passed to SCTVA. Following transformation and validation, DIWO presents the resulting code for final developer review and acceptance or rejection.

To support this decision process, DIWO provides a status-aware repository view that maintains file-level traceability across detected smells, rejected findings, rejected planning actions, and successfully refactored files. Developers can inspect affected source files and compare the original and refactored code before making the final decision. The orchestration layer also records workflow states, agent outputs, developer decisions, validation outcomes, and rollback actions in an audit trail. Accepted results can subsequently be delivered as a downloadable project artifact or committed to a separate source-code repository. This design preserves developer authority over design-sensitive decisions while automating the analysis, planning, transformation, and validation workflow.

F. Implementation Overview

The agents are implemented as independent Python-based services and communicate through REST interfaces and JSON contracts. CUQA integrates static-analysis and source-parsing techniques for repository inspection. RDP combines a refactoring knowledge base, software-metric features, a machine-learning prediction layer, MCDA ranking, and dependency ordering. SCTVA applies controlled AST/CST-based transformations and executes structural, test-based, behavioral, and invariant validation. DIWO acts as the workflow-control layer between the developer and the three technical agents: CUQA, RDP, and SCTVA. It introduces three explicit approval gates: code-smell selection, refactoring-plan approval, and final refactoring acceptance. Rejected artifacts are retained for traceability but are not forwarded to the next stage.

At the first gate, DIWO presents the structured CUQA code-quality report, including detected smells, affected files or source entities, and available impact information. The developer can approve or reject each finding, and only the approved code smells are passed to RDP.

At the second gate, DIWO presents the structured refactoring plan generated by RDP, including the selected refactoring method, expected impact, and planning information. Only developer-approved planning actions are transferred to SCTVA for transformation.

After SCTVA completes transformation and validation, DIWO provides a final review through a *status-aware hierarchical repository view*. Files are classified by workflow outcome, including detected smells, user-rejected findings, rejected planning actions, and successfully refactored files. The developer can inspect source-level changes using a before-after comparison together with validation evidence before accepting or rejecting the result.

Accepted results can be downloaded as a ZIP archive or published to a separate source-code repository with the

accepted files committed. DIWO also records workflow decisions and outcomes in a final audit view and downloadable audit report. Developer feedback from the approval stages is persisted for the feedback-learning component to model developer acceptance patterns and support future workflow assistance.

IV. EXPERIMENTAL DESIGN AND EVALUATION

The evaluation is organized around the four research responsibilities of the framework: analysis quality, planning quality, transformation safety, and human acceptance. The final experiment section should report both component-level results and the overall effect of the accepted refactorings on software quality.

A. Repositories and Test Cases

The study uses publicly available Java Python and C repositories that exhibit maintainability problems such as long methods, large classes, excessive parameters, duplicated logic, high complexity, and weak test coverage etc. Controlled test cases are also included so that known smell instances and expected transformation behavior can be verified. For the RDP learning component, refactoring examples are represented using structural metrics, smell labels, candidate actions, quality-improvement indicators, and risk-related features.

B. Evaluation Metrics

CUQA is evaluated using code-smell precision, recall, F1-score, complexity-identification accuracy, and processing time. RDP is evaluated using candidate-selection accuracy, plan executability, dependency-order correctness, skipped-smell rate, planning time, and the agreement between generated plans and expert/developer decisions. Where learned prediction is used, classification metrics such as precision, recall, and F1-score and regression errors such as mean absolute error are reported as appropriate.

SCTVA is evaluated using syntax-validation pass rate, structural-validation pass rate, behavioral-fingerprint agreement, invariant-preservation rate, test pass rate where tests exist, rollback accuracy, false-safe/false-unsafe decisions, validation time, and confidence-score behavior. DIWO is evaluated using developer approval/rejection patterns, decision time, rollback usage, audit completeness, perceived workflow clarity, trust in automated decisions, and usability measures. End-to-end effectiveness is measured using changes in code-smell count, cyclomatic complexity, duplication, coupling/cohesion-related indicators, maintainability index, and the proportion of accepted transformations that complete the full validation workflow.

C. Baseline Comparison

The framework should be compared against relevant baselines rather than evaluated in isolation. CUQA findings can be compared with established static-analysis outputs. RDP can be compared with direct smell-to-refactoring mapping that does not use ML, MCDA, or dependency-aware planning. SCTVA

can be compared with syntax/test-only validation to determine the additional value of behavioral fingerprinting and invariant mining. The end-to-end workflow can be compared with a fully automatic path without developer approval to examine the effect of human-in-the-loop control on acceptance and rollback decisions.

V. RESULTS AND DISCUSSION

A. Code Analysis Performance

The final analysis should first establish whether the quality report supplied to downstream agents is reliable enough for automated planning. Results should therefore be reported per smell category and per language rather than using only an overall detection percentage.

B. Planning Quality

RDP results should show whether the hybrid planner improves the suitability and executability of refactoring plans compared with direct rule mapping. Particular attention should be given to cases where dependency ordering changes the initial candidate sequence or where the planning stage deliberately skips a smell because preconditions are not satisfied.

C. Transformation Safety

The SCTVA analysis should distinguish between transformations that are structurally valid and transformations that preserve observed behavior. This distinction is central to the study because a transformation can compile successfully while still changing return values, exception behavior, or other runtime properties.

D. Developer Interaction and End-to-End Effect

The end-to-end discussion should connect technical quality improvements with developer decisions. A refactoring that produces a favorable metric change but is repeatedly rejected by developers should not be interpreted in the same way as a transformation that is both technically validated and accepted in the workflow.

VI. THREATS TO VALIDITY

Several factors can affect the interpretation of the results. Static smell detection is sensitive to thresholds and may produce false positives when project-specific design choices are not represented by general rules. The planning component depends on the quality and representativeness of its training and evaluation data, particularly for learned suitability and risk prediction. Behavioral fingerprinting and invariant mining provide execution-based evidence rather than a formal proof of semantic equivalence; unobserved inputs may still expose behavioral differences. Public open-source repositories may not represent the constraints of proprietary codebases, and developer acceptance can vary according to experience and project context. These threats are reduced by combining multiple repositories, reporting language- and smell-specific results, using controlled fault cases, comparing against base-lines, and separating structural validity from observed behavior preservation.

VII. CONCLUSION

This paper presents an agentic intelligent code refactoring assistant that treats code refactoring as a staged decision process rather than a single automatic code-generation task. CUQA transforms repository evidence into prioritized quality reports; RDP converts those reports into dependency-aware machine-executable plans; SCTVA applies approved transformations and combines structural, test-based, behavioral, and invariant validation with rollback; and DIWO keeps the developer involved through approval, comparison, feedback, and audit logging. The resulting architecture connects automation, safety, explainability, and human control within one refactoring workflow.

Future work will extend the evaluation to larger and more diverse repositories, improve the learned planning models using validated refactoring histories and developer feedback, and strengthen behavior-preservation analysis for projects with complex external dependencies and limited executable test harnesses.

REFERENCES

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley Professional, 2018.
- [2] T. Mens and T. Tourwé, "A survey of software refactoring," *IEEE Transactions on Software Engineering*, vol. 30, no. 2, pp. 126–139, 2004, doi: 10.1109/TSE.2004.1265817.
- [3] SonarSource, "Metrics definition," *SonarQube Server Documentation*, 2026. [Online]. Available: <https://docs.sonarsource.com/sonarqube-server/user-guide/code-metrics/metrics-definition>. [Accessed: Sep. 2, 2026].
- [4] E. A. AlOmar, M. W. Mkaouer, C. D. Newman, and A. Ouni, "On preserving the behavior in software refactoring: A systematic mapping study," *Information and Software Technology*, vol. 140, Art. no. 106675, 2021, doi: 10.1016/j.infsof.2021.106675.
- [5] M. A. Karabiyik, "RefactorGPT: A ChatGPT-based multi-agent framework for automated code refactoring," *PeerJ Computer Science*, vol. 11, Art. no. e3257, 2025, doi: 10.7717/peerj-cs.3257.
- [6] K. Oueslati, M. Lamothe, and F. Khomh, "RefAgent: A multi-agent LLM-based framework for automatic software refactoring," *arXiv:2511.03153*, 2025, doi: 10.48550/arXiv.2511.03153.
- [7] B. Šarenac, N. Anquetil, S. Ducasse, and P. Tesone, "A new architecture reconciling refactorings and transformations," *Journal of Computer Languages*, vol. 80, Art. no. 101273, 2024, doi: 10.1016/j.col.2024.101273.
- [8] N. Tsantalis, A. Ketkar, and D. Dig, "RefactoringMiner 2.0," *IEEE Transactions on Software Engineering*, vol. 48, no. 3, pp. 930–950, Mar. 2022, doi: 10.1109/TSE.2020.3007722.
- [9] J. L. Overbey and R. E. Johnson, "Differential precondition checking: A lightweight, reusable analysis for refactoring tools," in *2011 26th IEEE/ACM International Conference on Automated Software Engineering*, 2011, pp. 303–312, doi: 10.1109/ASE.2011.6100067.
- [10] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The Daikon system for dynamic detection of likely invariants," *Science of Computer Programming*, vol. 69, nos. 1–3, pp. 35–45, Dec. 2007, doi: 10.1016/j.scico.2007.01.015.
- [11] S. Segura, G. Fraser, A. B. Sánchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, Sep. 2016, doi: 10.1109/TSE.2016.2532875.
- [12] OpenRewrite, "Introduction to OpenRewrite," 2026. [Online]. Available: <https://docs.openrewrite.org/>. [Accessed: Sep. 2, 2026].
- [13] JetBrains, "Code refactoring," *IntelliJ IDEA 2026.2 Help*, 2026. [Online]. Available: <https://www.jetbrains.com/help/idea/refactoring-source-code.html>. [Accessed: Sep. 2, 2026].
- [14] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.